

# UNIVERSIDAD EAFIT

Análisis de Algoritmos

## Informe técnico - Práctica Final Integradora

Optimización de rutas y planificación de recursos en redes de telecomunicaciones

**Repositorio:** ADA\_PF\_GARCIA\_JIMENEZ\_USUGA

**Integrantes:** Mateo Usuga Vargas, Juan José García Urrego, Juan Esteban Jiménez Cardona

**Fecha:** 08 de mayo de 2026

## 1. Introducción

El proyecto integra tres paradigmas algorítmicos en un problema de planeación de infraestructura de telecomunicaciones. Cada registro del dataset Telco Customer Churn se interpreta como una solicitud de servicio; estas solicitudes se ordenan y consultan con Divide y Vencerás, alimentan una asignación de ancho de banda con Programación Dinámica y también sirven para construir una topología sintética de red sobre la cual se aplica Kruskal como algoritmo codicioso.

MergeSort es apropiado para ordenar por prioridad porque conserva estabilidad y ofrece tiempo garantizado  $O(n \log n)$  en el peor caso. Kruskal es apropiado para el tendido mínimo de cableado porque el problema corresponde a un árbol de expansión mínima. La Mochila 0-1 requiere Programación Dinámica porque elegir la solicitud de mejor razón valor/peso no garantiza el óptimo global.

## 2. Descripción del dataset

Se cargó el archivo WA\_Fn-UseC\_-Telco-Customer-Churn.csv. Del CSV se extrajeron customerID, tenure, MonthlyCharges, TotalCharges y Churn. Los TotalCharges en blanco, asociados a tenure = 0, se trataron como 0.0.

| Propiedad               | Valor     |
|-------------------------|-----------|
| Registros cargados      | 7043      |
| TotalCharges nulos      | 11        |
| Churn = No              | 5174      |
| tenure máximo           | 72        |
| tenure mínimo           | 0         |
| MonthlyCharges promedio | 64.76 USD |

Para el grafo, los 7043 registros se distribuyeron en 20 grupos mediante  $i \bmod 20$ . Luego se calculó el promedio de MonthlyCharges por grupo, redondeado a dos decimales, y el costo de cada arista  $(u,v)$  se definió como  $\text{piso}(\text{promedio}_u + \text{promedio}_v)$ . Esto genera un grafo completo con 20 nodos y 190 aristas.

## 3. Módulo A - Divide y Vencerás

MergeSort ordena las solicitudes por tenure descendente. En la mezcla, ante empates se toma primero el elemento de la mitad izquierda, por lo que el algoritmo es estable y conserva el orden original de solicitudes con el mismo tenure.

Pseudocódigo de Merge Sort:

```
MergeSort(A, izq, der):  
    si izq >= der:  
        retornar  
  
    mid = izq + (der - izq) / 2  
    MergeSort(A, izq, mid)  
    MergeSort(A, mid+1, der)  
  
    Merge(A, izq, mid, der)
```

Pseudocódigo de Merge:

```
Merge(A, izq, mid, der):  
    L = A[izq..mid]  
    R = A[mid+1..der]
```

```

i = 0, j = 0, k = izq
mientras i < |L| y j < |R|:
    si L[i].tenure >= R[j].tenure:
        A[k] = L[i]
        i++
    sino:
        A[k] = R[j]
        j++
    k++
mientras i < |L|:
    A[k++] = L[i++]
mientras j < |R|:
    A[k++] = R[j++]

```

La recurrencia  $T(n)=2T(n/2)+\Theta(n)$  se resuelve como  $\Theta(n \log n)$  por el teorema maestro. La búsqueda binaria recursiva se implementó sobre el arreglo ordenado previamente para encontrar un registro que tenga un tenure igual a k. Si existen varios registros con el mismo valor, se retorna cualquiera. Si no existe ninguno, la función retorna -1.

```

binarySearch(A, izq, der, k):
    si izq > der:
        retornar -1
    mid = izq + (der - izq) / 2
    si A[mid].tenure == k:
        retornar mid
    si A[mid].tenure > k:
        retornar binarySearch(A, mid+1, der, k)
    retornar binarySearch(A, izq, mid-1, k)

```

| n    | tiempo ms | n log2 n  | tiempo/(n log n) |
|------|-----------|-----------|------------------|
| 1000 | 2.5247    | 9,965.78  | 0.000253         |
| 3500 | 9.421     | 41,205.99 | 0.000229         |
| 7043 | 20.9581   | 90,023.45 | 0.000233         |

| Consulta | k  | customerID |
|----------|----|------------|
| Q_A01    | 72 | 6904-JLBGY |
| Q_A02    | 60 | 7426-WEIJX |
| Q_A03    | 45 | 7925-PNRGI |
| Q_A04    | 30 | 2684-EIWEO |
| Q_A05    | 12 | 7450-NWRTR |

## 4. Módulo B - Algoritmo codicioso

El grafo generado contiene 20 nodos, 190 aristas y costo promedio de arista 129.02. Kruskal ordena las aristas por costo y agrega una arista solamente si conecta dos componentes diferentes. La estructura Union-Find aplica compresión de caminos y unión por rango.

```

Kruskal(G):
    ordenar aristas por peso ascendente
    crear UnionFind(V)
    MST = vacio
    para cada arista (u,v,w):
        si find(u) != find(v):
            unir(u,v)
            agregar (u,v,w) a MST
    retornar MST

```

Peso total del MST obtenido: 2388. La arista concreta usada para ilustrar el lema del ciclo fue (12, 16, 123): al evaluarla, sus extremos ya estaban conectados por aristas previamente seleccionadas, por lo que agregarla formaba un ciclo. Como Kruskal procesa en orden no decreciente, esa arista no puede mejorar el árbol y puede descartarse sin perder optimalidad.

| #  | Arista (u,v,costo) |
|----|--------------------|
| 1  | (1, 12, 122)       |
| 2  | (1, 16, 123)       |
| 3  | (0, 1, 124)        |
| 4  | (1, 9, 124)        |
| 5  | (1, 14, 124)       |
| 6  | (1, 7, 125)        |
| 7  | (1, 11, 125)       |
| 8  | (1, 13, 125)       |
| 9  | (1, 19, 125)       |
| 10 | (1, 3, 126)        |
| 11 | (1, 10, 126)       |
| 12 | (1, 17, 126)       |
| 13 | (1, 18, 126)       |
| 14 | (1, 2, 127)        |
| 15 | (1, 4, 127)        |
| 16 | (1, 5, 127)        |
| 17 | (1, 8, 127)        |
| 18 | (1, 6, 129)        |
| 19 | (1, 15, 130)       |

Verificación manual del subgrafo inducido por los primeros 5 nodos: se seleccionaron (0, 1, 124), (1, 3, 126), (1, 2, 127), (1, 4, 127) con peso total 504. Kruskal produjo el mismo conjunto y peso.

## 5. Módulo C - Programación Dinámica

La recurrencia de Mochila 0-1 usada fue:  $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$  si  $w_i \leq w$ ; en caso contrario  $dp[i][w] = dp[i-1][w]$ . La tabla tiene dimensiones  $(n+1)$  por  $(W+1)$ , y el backtracking compara  $dp[i][w]$  contra  $dp[i-1][w]$  para reconstruir las solicitudes seleccionadas.

```

Mochila(items, W):
    crear dp[0..n][0..W] = 0
    para i = 1..n:
        para w = 0..W:
            dp[i][w] = dp[i-1][w]
            si peso[i] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w-peso[i]] + valor[i])
    reconstruir desde dp[n][W] hacia dp[0][*]

```

Bajo la especificación literal, las 50 solicitudes activas de mayor tenure tienen pesos entre 1363 y 8468, todos mayores que  $W=500$ . Por tanto, el valor óptimo literal es 0 y se seleccionan 0 solicitudes. Esta inconsistencia impide construir un contraejemplo codicioso dentro de ese conjunto exacto.

Como verificación complementaria y reproducible, se tomó el mismo orden pero filtrando solicitudes activas con  $0 < \text{peso} \leq 500$  hasta reunir 50. En ese conjunto la PD obtuvo valor 249, peso total 493 y seleccionó: 9092-GDZKO.

| customerID | tenure | peso | valor | ratio  |
|------------|--------|------|-------|--------|
| 9950-MTGYX | 28     | 488  | 203   | 0.4160 |
| 5171-EPLKN | 27     | 470  | 199   | 0.4234 |
| 7956-XQWGU | 27     | 460  | 194   | 0.4217 |

| Enfoque               | Selección  | Valor total | Óptimo |
|-----------------------|------------|-------------|--------|
| Codicioso (ratio v/w) | 5171-EPLKN | 199         | No     |
| PD (Mochila 0-1)      | 9950-MTGYX | 203         | Sí     |

La complejidad temporal y espacial es  $\Theta(n \cdot W)$ . Este método es pseudopolinomial, porque depende del valor numérico de  $W$  y no solo del número de bits necesarios para representarlo.

## 6. Integración del pipeline

El programa se ejecuta con el comando indicado en el README y produce todos los archivos de results/ en una sola ejecución. El flujo completo es:

```
CSV -> Parser -> Solicitudes originales
      |-> Modulo A: MergeSort + busqueda -> solicitudes_ordenadas.csv, busquedas_A.txt
      |-> Modulo B: grupos i mod 20 -> grafo completo -> Kruskal -> mst_red.txt
Solicitudes ordenadas -> Modulo C: seleccion activa -> Mochila 0-1 -> asignacion_bw.txt
```

El Módulo C depende directamente de la salida del Módulo A, porque usa el arreglo ordenado por tenure para tomar las solicitudes activas de mayor prioridad. El Módulo B usa el dataset original para conservar la asignación determinista de grupos por posición  $i \bmod 20$ .

## 7. Conclusiones

MergeSort ofrece garantías de ordenamiento estable con costo  $O(n \log n)$ , adecuado para priorizar solicitudes sin perder el orden original en empates. Kruskal ofrece optimalidad global para el MST gracias al lema del ciclo y al uso de Union-Find para detectar ciclos eficientemente. La Mochila 0-1 evidencia que una decisión local por ratio puede fallar, por lo que la Programación Dinámica es necesaria cuando las combinaciones factibles compiten por una capacidad limitada.

Los datos reales también muestran un punto importante: una especificación teórica puede volverse inconsistente al aplicarse directamente al dataset. En este caso, los pesos de las 50 solicitudes activas de mayor tenure superan la capacidad  $W=500$ , lo cual deja una solución literal vacía. Por eso se documentó el resultado literal y se agregó una ejecución complementaria reproducible para evidenciar el comportamiento esperado de la PD y el fallo codicioso.

## 8. Herramientas utilizadas

Se usó C++17 para la implementación, std::chrono para las mediciones, y Python/ReportLab para construir el PDF inicial a partir de los resultados generados. También se usó ChatGPT como apoyo para revisión del enunciado, detección de inconsistencias, depuración de compilación y redacción asistida del informe; Por último, también se usó Claude para resolver dudas, bugs y revisar tanto el código como el informe.

## 9. Referencias

Kaggle. Telco Customer Churn dataset, CC0 Public Domain. URL indicada en el enunciado de la práctica.

Cormen, Leiserson, Rivest y Stein. Introduction to Algorithms. MIT Press.

Material del curso Análisis de Algoritmos, Universidad EAFIT, 2026/01.

## 10. Roles del equipo

| Integrante                   | Contribución principal                                                        |
|------------------------------|-------------------------------------------------------------------------------|
| Mateo Usuga Vargas           | Módulo A: parser, MergeSort, búsqueda binaria y mediciones empíricas.         |
| Juan José García Urrego      | Módulo B: construcción del grafo, Kruskal, Union-Find y verificación del MST. |
| Juan Esteban Jiménez Cardona | Módulo C: Mochila 0-1, backtracking, contraejemplo y análisis de complejidad. |